PROJECT SYNOPSIS & DESIGN REPORT

Multi-Asset Portfolio Tracker & Real-Time Analytics Dashboard

Subject: Software Project Engineering & Design
Date of Submission: July 29, 2026
Author: Software Development Team
Stack: Flask Web Server, SQLite3 DB, JS (GSAP, Chart.js)

1. Project Synopsis (Project Proposal)

In the modern personal finance landscape, retail investors face significant difficulties in managing their investments due to portfolio fragmentation. Individuals hold diversified assets across multiple brokerage platforms—including equity holdings, exchange-traded funds (ETFs), cash positions, and cryptocurrencies. Keeping track of consolidated net worth across separate accounts creates high administrative overhead, latency in risk assessment, and poor allocation visibility.

The **Proposed Multi-Asset Portfolio Tracker** solves these challenges by consolidating all assets into a single, high-performance web dashboard. The system automatically fetches and renders real-time valuations, calculates active yields (unrealized profit/loss, percentage gains), assesses overall portfolio concentration weights, and maps external market news feeds directly to user holdings.

Key Objectives:

- **Data Consolidation:** Create a single database repository for user assets.
- **Real-Time Sync:** Query Yahoo Finance and CoinGecko dynamically via custom backend proxy routes.
- **Risk Assessment:** Highlight concentration risk metrics, top asset exposure percentages, and diversification ratings.
- **Local AI Assistance:** Embed an AI Portfolio Analyst agent providing direct analytical feedback on holding structures.

Metric	Existing System	Proposed System
Net Worth tracking	Manual copy-pasting across various brokerage sites.	Unified SQL DB syncing all portfolios.
Asset Updating	Stale values, manual entries of end-of-day quotes.	Real-time live synchronization (Yahoo/CoinGecko).
CORS API blocks	CORS blocks prevent querying external APIs directly.	Flask-based proxy route rewrites and handles URLs.
Analytics	Static sheets lacking risk analysis warnings.	Active risk profile grading and concentration metrics.

2. Requirement Specification

Hardware Requirements:

- **CPU:** Dual-Core Processor 2.0 GHz or higher (Intel Core i3/i5/i7 or equivalent).
- **RAM:** 4 GB minimum (8 GB recommended for fluid development).
- **Disk Storage:** 250 MB of free hard drive space for packages and DB logs.
- **Network Connection:** Active internet connection required for API price fetches.

Software Requirements:

- **Operating System:** Windows 10/11, macOS 10.15+, or Linux Ubuntu 20.04+.
- **Runtime Environment:** Python 3.10+.
- **Backend Web Framework:** Flask 3.0+.
- **Database Engines:** SQLite3 (Relational, serverless database engine).
- **Frontend Stack:** HTML5, CSS3, JavaScript (ES6).
- **External Libraries:** Chart.js (for Canvas graphs) and GSAP (for UI stagger animations).

3. Feasibility Study

A. Technical Feasibility

The project is technically feasible. The implementation of custom proxy endpoints handles browser CORS blocks securely by routing queries through Python's built-in urllib module. Chart.js and GSAP are lightweight client-side scripts, ensuring zero performance hits. The serverless SQLite3 structure simplifies database maintenance.

B. Economic Feasibility

The project relies entirely on open-source libraries, runtimes, and databases, resulting in zero development licensing costs. Furthermore, it accesses free public APIs (Yahoo Finance and CoinGecko), completely eliminating the need for expensive monthly financial data subscriptions. Operational deployment fits easily within free tiers.

C. Organizational & Cultural Feasibility

The app integrates smoothly into a user's daily financial workflows. It presents a clean, intuitive layout designed specifically for retail investors (supporting local formatting like INR currency notations and Indian market RSS feeds). It maintains complete user data privacy through local storage configurations.

4. Event Table

The application follows an event-driven flow where user interface interactions trigger backend routes and update state arrays. Below is a structured event model of the system:

Event Trigger	Source	Action / Process	Output State
Registration Form Submitted	Auth Modal	Hashes password; inserts record into USERS table.	User registered; returns HTTP 200.
Login Form Submitted	Auth Modal	Validates password hash; generates session cookie.	Loads portfolio workspace view.
Add/Edit Asset Form Submitted	Holdings Modal	Computes buy value; guesses initial price; saves in HOLDINGS.	Recalculates totals; updates table UI.
Refreshes Dashboard screen	Browser event	Fetches CoinGecko & Yahoo Finance prices via /proxy/.	Updates CMP; recalculates gains/losses.
AI Chat Prompt sent	Analyst Chat	Prepares portfolio statistics; calls backend local AI agent.	Appends analyst response to chat list.

5. Entity-Relationship (ER) Diagram

The relational database schema uses a streamlined relational structure built inside SQLite3. The diagram displays a one-to-many cardinality between Users and Holdings.

Entity	Attributes & Keys	Relationship / Card.
USERS	■ email (PK) # password_hash # created_at	One User can own 0 to many Holdings (1 : N)
HOLDINGS	■ id (PK) ■ user_email (FK) # symbol # exchange # name # yahooSymbol # assetClass # qty # buyPrice # price	Many Holdings link to a single registered User email.

6. System Design Details (UML Models)

A. Use Case Diagram

Actor: Registered Investor
Use Cases:
• **Manage Holdings:** Allows the user to add new asset purchases, edit quantities/buy prices, and record sales.
• **View Portfolio Analytics:** Renders Chart.js and list summaries of asset weights and risk ratings.
• **Aggregate Stock News:** Synchronizes RSS feeds to provide stock news based on portfolio symbols.
• **Query AI Analyst:** Evaluates portfolio holdings and answers investment strategy questions.

B. Class Diagram

The system classes are organized as follows:
1. **DatabaseManager:** Handles connection sessions, queries, and writes to database.db.
2. **FlaskController:** Maps REST endpoints (`/api/holdings`, `/api/news`, `/proxy/`, `/api/ask-ai`) to their processing handlers.
3. **UIEngine:** Operates client-side logic, holds UI states, parses JSON data, and updates DOM trees.

C. Activity Diagram (Live Price Fetch Workflow)

The workflow steps when the user triggers the Live Price update:
1. Initialize live price caching lists.
2. Loop through holdings and fetch Crypto prices directly from CoinGecko API.
3. Filter out stocks, checking if they are Indian Equities (`NSE`/`BSE`).
4. If Indian, query Yahoo Finance proxy endpoint (`/proxy/https://query1.finance.yahoo.com/...`).
5. If US/Global, query Twelve Data API endpoint.
6. If any fetch fails, retrieve fallback data or simulated open-market drift figures.
7. Recalculate portfolio parameters, render updated values, and persist updates back into database.db.